Predicting Video Game Sales and Genre Using Machine Learning

Adrian Lara
Information Technologies for Sciences
National Autonomous University of Mexico (UNAM)
Morelia, Mexico
adrianlarasc@gmail.com

Abstract

This project applies three supervised machine learning algorithms: Decision Tree, Logistic Regression, and Naive Bayes to the Video Game Sales dataset (vgsales) sourced from Kaggle. The dataset was previously engineered with additional features including `Publisher_Tier`, `Genre_Group`, `Platform_Gen`, `Sales_Tier`, and `Decade`, which we will use for our analysis. The Decision Tree and Logistic Regression models were trained to predict `Sales_Tier`, while the Naive Bayes model was trained to predict `Genre_Group`. While Decision Tree and Logistic Regression achieved results that may seem low, they provided valuable insights on the nature of the data and the impact of class imbalance on an extremely right-skewed dataset. Naive Bayes performed poorly, which can be attributed to the violation of its independence assumption. Class imbalance was identified as a central challenge, but I still wanted to explore its effects with this project, as this dataset also gave a very interesting statistics project. For reference, here's a view of the distribution of the sales in this dataset, along with the distribution of the transformed Sales variable using log1p:

ACM Reference Format:
Adrian Lara. 2026. Predicting Video Game Sales and Genre Using Machine Learning. In *Proceedings of Machine Learning Course Project (ML Course Project)*. ACM, New York, NY, USA, 3 pages.

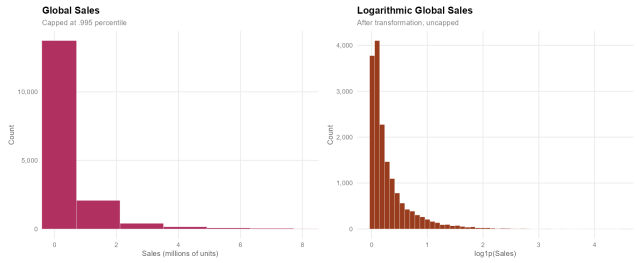


Figure 1: Distribution of `Global_Sales` (left) and log-transformed `Global_Sales` (right).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ML Course Project, UNAM, Morelia, Mexico
© 2026 Copyright held by the owner/author(s).

1 Introduction

The video game industry is a multi-billion dollar market with a highly competitive landscape. Predicting the sales performance of videogames is a highly complex task influenced by a multitude of factors. In this project, we explore the application of three supervised machine learning models to determine how well we can predict a game's sales tier and genre based on a set of engineered features like decade, platform generation, and publisher tier.

The dataset used is the Video Game Sales dataset (vgsales) from Kaggle, which contains over 16,000 entries with regional and global sales figures for games released between the 1980s and the 2010s.

Three classification algorithms were selected for this study: Decision Tree, Logistic Regression, and Naive Bayes. Each model was chosen for its interpretability. A key objective of this project is not only to compare model accuracy, but to understand *why* each model behaves the way it does, including its failures.

2 Dataset and Feature Engineering

2.1 Original Dataset

The raw vgsales dataset contains the following columns: `Rank`, `Name`, `Platform`, `Year`, `Genre`, `Publisher`, `NA_Sales`, `EU_Sales`, `JP_Sales`, `Other_Sales`, and `Global_Sales`. Sales values are expressed in millions of units sold.

Missing values were handled in the preprocessing step (written in R): the 271 missing `Year` values were imputed with the median release year of the game's platform, and the 58 missing `Publisher` values were labelled `"Unknown"`.

2.2 Engineered Features

Five new variables were derived from the original columns to improve model performance and interpretability.

Publisher_Tier groups the original 578 publishers by number of games into three categories: *Major* (top 10), *Mid* (ranks 11–50), and *Small* (the rest). This reduces sparsity.

Genre_Group consolidates the original 12 genres into the top 7 most frequent genres plus an *Other* category.

Platform_Gen maps each platform to its hardware generation, ranging from Gen3-4 through Gen8, plus a *PC/Other* category for PC and niche platforms.

Sales_Tier bins `Global_Sales` into four ordinal categories: *Niche*, *Moderate*, *Hit*, and *Blockbuster*. Niche was engineered according to the games below the median, while the others were engineered according to personal perception of the sales tiers.

Decade derives the release decade from the `Year` column, yielding four categories: 1980s, 1990s, 2000s, and 2010s.

2.3 Class Imbalance

A significant class imbalance was identified in `Sales_Tier`. The vast majority of games fall into the *Niche* and *Moderate* categories, while *Hit* accounts for very little results along *Blockbuster*. This imbalance poses a challenge for model training.

3 Methodology

3.1 Experimental Setup

All three models were trained and evaluated using Python's `scikit-learn` library. The dataset was split into 80% training and 20% testing using stratified sampling to preserve class proportions in both sets (`random_state=42`). Hyperparameter tuning was performed using `GridSearchCV` with 5-fold cross-validation on the training set.

3.2 Decision Tree

The Decision Tree classifier was trained to predict `Sales_Tier` using `Publisher_Tier`, `Genre_Group`, `Platform_Gen`, and `Decade` as features. All features were ordinal-encoded inside a `Pipeline`. `GridSearchCV` was used to search over `max_depth` (1–20), `min_samples_split` (5, 10), and `criterion` (*gini*, *entropy*). Two trees were tuned: an unweighted tree selected by accuracy, and a tree with `class_weight="balanced"` selected by *balanced accuracy* (the mean recall over the four classes). Tuning the weighted tree by plain accuracy would defeat its purpose, since accuracy rewards predicting the majority class.

3.3 Logistic Regression

Logistic Regression was also trained to predict `Sales_Tier` on the same feature set, enabling direct comparison with the Decision Tree. Features were one-hot encoded via `OneHotEncoder` and scaled with `StandardScaler` inside a `Pipeline` to prevent data leakage. The regularization parameter `C` was tuned via `GridSearchCV` across six values (0.001 to 100) on a log scale, using balanced accuracy as the selection metric. Multinomial logistic regression was used with the `lbfgs` solver and `class_weight="balanced"`.

3.4 Naive Bayes

The Naive Bayes model was trained to predict `Genre_Group`. `CategoricalNB` was selected over `GaussianNB` because all features `Publisher_Tier`, `Platform_Gen`, `Sales_Tier`, and `Decade` are discrete categories with no continuous distribution. Features were encoded with `OrdinalEncoder`. The smoothing parameter `alpha` was tuned via `GridSearchCV` across seven values (0.01 to 10), using accuracy as the selection metric.

4 Results

4.1 Decision Tree

Without class weighting, the best tree (`max_depth=9`, *gini*) achieved 56% accuracy but predicted *Blockbuster* with zero recall, effectively ignoring the rarest class, and recalled only 7% of *Hit* games. With `class_weight="balanced"` (best `max_depth=11`), overall accuracy dropped to 40%, but balanced accuracy rose from 0.33 to 0.41: recall for *Blockbuster* improved from 0.00 to 0.54 and recall for *Hit* from 0.07 to 0.33.

The decision tree visualization below shows the first three levels of the balanced tree; its first split is on `Publisher_Tier`. In the unweighted tree, `Publisher_Tier` (importance 0.41) and `Platform_Gen` (0.28) were the most influential predictors. In the balanced tree the importances are more evenly spread (`Genre_Group` 0.29, `Publisher_Tier` 0.29, `Platform_Gen` 0.25, `Decade` 0.17), since it also has to separate the rare classes.

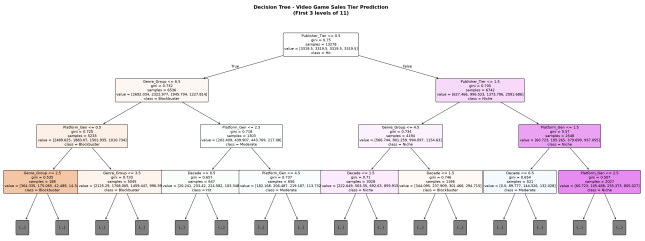


Figure 2: Decision Tree visualization (first 3 levels).

4.2 Logistic Regression

Logistic Regression with `class_weight="balanced"` and the best `C` selected by `GridSearchCV` (`C=0.01`) produced results comparable to the balanced Decision Tree: 40% accuracy and 0.41 balanced accuracy. It recalled 68% of *Blockbuster* games but only 16% of *Hit* games, and it frequently over-predicts *Blockbuster* (low precision). Table 1 summarizes the test-set metrics of all models.

Table 1: Test-set metrics. Naive Bayes predicts a different target (8 genre groups), so it is not directly comparable.

Model	Accuracy	Bal. acc.	Macro F1
Decision Tree	0.565	0.331	0.325
Decision Tree (balanced)	0.396	0.406	0.300
Logistic Regression (balanced)	0.399	0.411	0.287
Naive Bayes (genre)	0.276	0.181	0.146

The confusion matrix in the following figure shows the distribution of correct and incorrect predictions across all four classes.

4.3 Naive Bayes

The Naive Bayes model (best `alpha=5`) achieved 28% accuracy on `Genre_Group` prediction, with four of the eight genre classes receiving zero recall (*Misc*, *Racing*, *Role-Playing*, *Shooter*). The confusion matrix is shown in Figure 4.

5 Discussion

5.1 Class Imbalance

Class imbalance was the dominant challenge across all three models. In both the Decision Tree and Logistic Regression, applying `class_weight="balanced"` forced the models to give equal penalty to misclassifying rare classes at the cost of overall accuracy. Without weighting, a model trained on an imbalanced dataset tends to just learn to predict the majority class. For the same reason, the

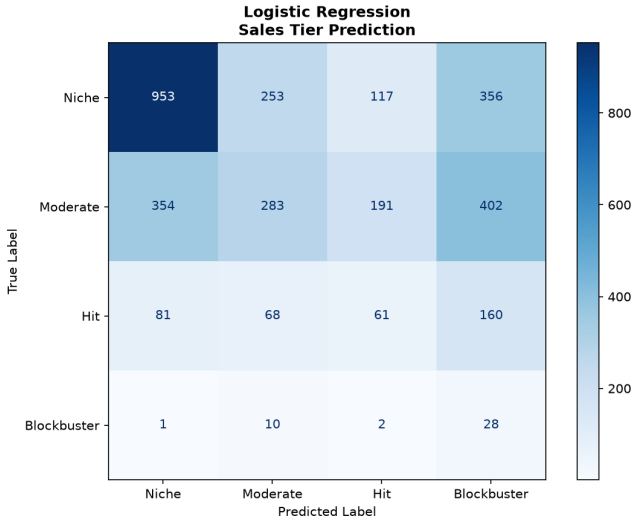


Figure 3: Confusion matrix for Logistic Regression.

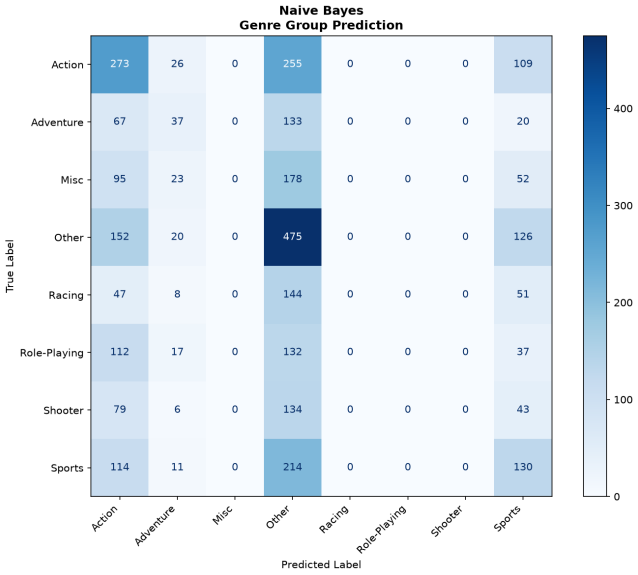


Figure 4: Confusion matrix for Naive Bayes.

weighted models were tuned with balanced accuracy instead of plain accuracy.

6 Conclusions

This project trained and evaluated three machine learning classifiers on the vgsales dataset. The Decision Tree and Logistic Regression demonstrated moderate ability to predict `Sales_Tier` from engineered categorical features, with class weighting improving fairness at the cost of overall accuracy. Naive Bayes achieved low accuracy on genre prediction, primarily due to violated independence assumptions.

The results underscore three broader lessons: (1) feature engineering can sometimes limit a model's performance; (2) accuracy alone is a misleading metric under class imbalance; and (3) a model that fails informatively is often more valuable for learning than one that succeeds by chance, as seen in this project. Personally, I found this project to be very interesting and informative.

References

[1] `Video Game Sales dataset. Kaggle.` https://www.kaggle.com/datasets/gregorut/videogamesales
[2] scikit-learn developers. `sklearn.preprocessing.LabelEncoder` 2025. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html
[3] scikit-learn developers. `sklearn.preprocessing.OneHotEncoder` 2025. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html
[4] scikit-learn developers. `sklearn.preprocessing.OrdinalEncoder` 2025. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html
[5] GeeksforGeeks. Training a decision tree against unbalanced data. *GeeksforGeeks - Machine Learning*, Nov. 2024. https://www.geeksforgeeks.org/machine-learning/training-a-decision-tree-against-unbalanced-data/
[6] GeeksforGeeks. Multiclass logistic regression. *GeeksforGeeks - Artificial Intelligence*, Aug. 2025. https://www.geeksforgeeks.org/artificial-intelligence/multiclass-logistic-regression/